DYSCO: Dynamic Attention-Scaling Decoding for Long-Context LMs

Authors: Ye, et al.
Presented by: Fengnan Li

Duke B&B

Mar, 2026

Outline

1. Background: Long-Context Challenges
   - Context rot and performance degradation
   - Lost in the Middle phenomenon
2. Background: Retrieval Heads
   - Definition, properties, and downstream impact
   - From Retrieval Heads to QRHead
3. DYSCO: Method
   - Motivation: attention dilution during decoding
   - Three-stage algorithm: Aggregation, Selection, Rescaling
4. Experiments & Analysis
5. Conclusion

The Long-Context Challenge

The Problem

Modern LMs support 128K+ tokens, but **accuracy degrades significantly** as input length grows — "**context rot**" (Hong et al., 2025).

Example: Path Traversal task:

- Given graph edges, find a path from source to target
- Structurally simple — just chain edges
- Qwen3-8B: **57% at 4K → 1% at 32K**
- Models fail well below their claimed context window

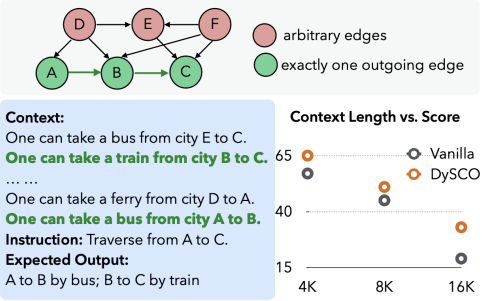


Fig 1 from DYSCO (Ye et al., 2026)

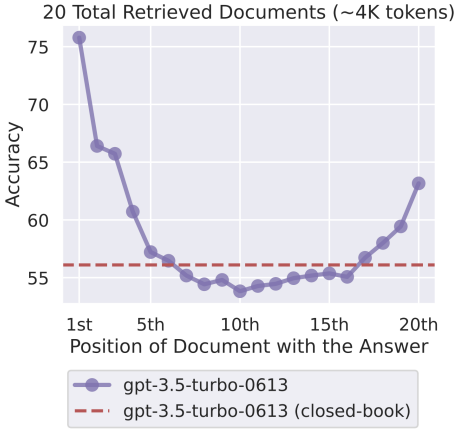
Lost in the Middle (Liu et al., 2023)

Key Finding: LLM performance shows a **U-shaped curve** based on the position of relevant information:

- **Primacy bias**: Best when info is at the beginning
- **Recency bias**: Good when info is at the end
- **Worst**: When info is in the middle

Implications:

- Multi-doc QA drops >20% when answer is in the middle
- Performance with 20 docs can be *worse* than closed-book
- Extended-context models are **not necessarily better** at using context



U-shaped performance: 20 docs (∼4K tokens). Position of answer document vs. accuracy.

Why Does This Happen? Attention Dilution

The Attention Dilution Problem

As context length grows, softmax attention distributes probability mass across **more tokens**, diluting the signal from task-relevant information.

Softmax Attention:

$$\alpha_{t,i} = \frac{\exp(q_t^\top k_i/\sqrt{d})}{\sum_{j=1}^{n} \exp(q_t^\top k_j/\sqrt{d})}$$

- Denominator grows with context length n
- Attention mass on relevant tokens **decreases**

Consequences:

- **Attention sinks**: Model dumps mass on initial tokens as "defaults"
- **Signal drowning**: Relevant tokens compete with thousands of irrelevant ones
- **Compounding errors**: Each reasoning step may attend to wrong context

$\Rightarrow$ **Challenge:** How to keep attention focused on relevant context?

What Are Retrieval Heads? (Wu et al., 2025)

Definition

Retrieval heads are a specialized subset of attention heads that perform **copy-paste** operations — redirecting tokens from the input context to the output during decoding.

Detection via Needle-in-a-Haystack:

- Insert a unique sentence (needle) in a long context (haystack)
- Ask the model to retrieve the needle
- Track which attention heads attend to the needle tokens during generation

Retrieval Score for head h:

$$\text{Retrieval_Score}(h) = \frac{|g_h \cap k|}{|k|}$$

where g_h = set of tokens copied by head h, k = needle tokens.

A score of 0.9 means the head copy-pastes 9 out of 10 needle tokens.

Five Key Properties of Retrieval Heads

1. **Universal**: Present in all model families tested (LLaMA, Yi, Qwen, Mistral)
2. **Sparse**: Only $<$**5%** of attention heads are retrieval heads
3. **Intrinsic**: Already exist in base pretrained models; preserved through fine-tuning and context extension
4. **Dynamically activated**: $\sim$12 strongest heads always active; weaker heads activated by specific contexts
5. **Causal**: Masking retrieval heads $\rightarrow$ **hallucination**; masking random heads $\rightarrow$ no effect

Masking Experiment (LLaMA-2-7B)

Mask top 20 retrieval heads:
NIAH accuracy: **63.6** (from $\sim$100)
$\rightarrow$ Severe hallucination

Mask 20 random heads:
NIAH accuracy: **94.7**
$\rightarrow$ Nearly no effect

Retrieval heads affect **factuality**, *not fluency — masked model says "go to the beach" instead of correct answer.*

Retrieval Heads: Impact on Downstream Tasks

Extractive QA

Masking 100 retrieval heads:
F1 score drops **56.7** → **32.3** (–43%)

Masking 100 random heads:
F1 score: **55.4** (–2%)

Real-world document QA heavily relies on retrieval heads.

Chain-of-Thought Reasoning

Masking retrieval heads severely impacts **CoT** tasks:

Task	No mask	Mask 100
GSM8K	45.1	**2.0**
MMLU	43.6	**24.8**

Without CoT, masking has **minimal** effect — model uses internal knowledge (FFN).

With CoT, model must **refer back** to input → needs retrieval heads.

From Retrieval Heads to QRHead (Zhang et al., 2025)

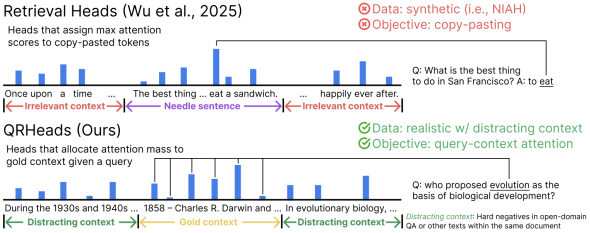


Figure 2: Comparison between Retrieval Heads (Wu et al., 2025b) and QRHEAD (Ours).

Fig 2 from QRHead (Zhang et al., 2025): Comparison of detection methods.

QRHead Improvement:

- **Query-context attention scoring**: Measures how much attention head h allocates from query to gold context
- Uses **realistic tasks** instead of synthetic NIAH
- Only needs <100 examples

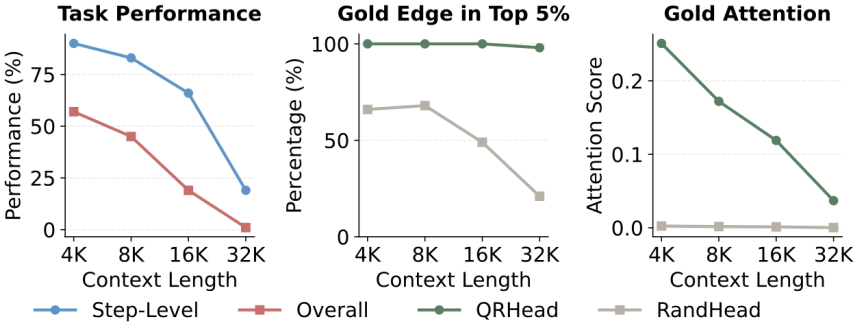
Only <**25% overlap** in top 32 heads between the two methods.

Masking QRHead causes **more severe** NIAH degradation than masking original retrieval heads.

$\Rightarrow$ **DYSCO adopts QRHead**.

Motivation: Retrieval Heads Stay Focused, But Get Diluted

Diagnostic: PATH TRAVERSAL on Qwen3-8B — find a 5-node path in a graph.



Key Insight: QRHead attention provides a reliable **relevance signal** for identifying important tokens, even as overall performance degrades. Can we leverage this signal to **steer all heads**?

DYSCO: Overview of the Three-Stage Algorithm

DYSCO: Dynamic Attention-Scaling Decoding

At each decoding step, DYSCO dynamically **up-weights** tokens that retrieval heads identify as relevant, by intervening on the attention logits of **all** heads.

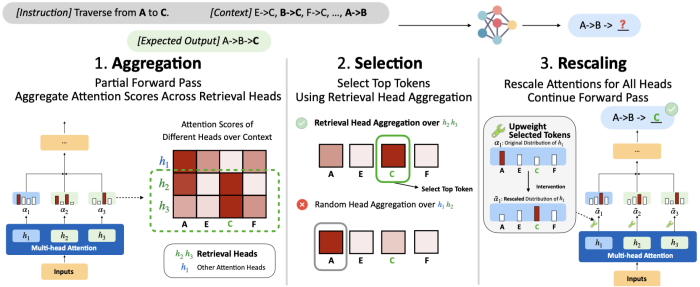


Figure 2. Overview of DYSCO algorithm. At each decoding step, DYSCO consists of three stages: (1) **Aggregation**: We run a partial forward pass over the input sequence to obtain attentions of retrieval heads, such as QRHEAD, and use them to assign relevance scores to context tokens; (2) **Selection**: We use the relevance scores to select the important tokens; (3) **Rescaling**: We up-weight the important tokens by intervening attention logits of all attention heads and run a full forward pass to sample the next token.

Training-free · Decoding-time only · Works on any off-the-shelf LM

DYSCO: Algorithm Details

Algorithm 1 DYSCO

Input: LM $\mathcal{M}$ with attention heads $\mathcal{H}$, QRHEAD heads $\mathcal{H}^*$, input $\boldsymbol{x}_{\leq T}$, rescale strength β, momentum γ, cumulative probability threshold p (for token selection).

Output: Generated sequence $\boldsymbol{x}_{\leq T'}$

$t \leftarrow T$

$\mathbf{r}_T \leftarrow \mathbf{r}_{\text{init}}$ `// Initialize Relevance`

while not finished **do**

  $t \leftarrow t + 1$

  (Aggregation) Run a partial forward pass to obtain attention logits from $\mathcal{H}^*$: $\mathbf{a}_t^{(h)}$ for $h \in \mathcal{H}^*$.

  $\mathbf{r}_t \leftarrow \frac{1}{|\mathcal{H}^*|} \sum_{h \in \mathcal{H}^*} \text{Softmax}(\mathbf{a}_t^{(h)})$

  $\mathbf{r}_t \leftarrow \gamma \cdot \mathbf{r}_{t-1} + (1 - \gamma) \cdot \mathbf{r}_t$ `// Apply Momentum`

  (Selection) $\boldsymbol{x}^* \leftarrow \text{SelectTop}(\boldsymbol{x}_{\leq t}, \mathbf{r}_t, p)$ `// Select top-p tokens`

  $\mathbf{v}[i] \leftarrow \begin{cases} \log(\beta) & x_i \in \boldsymbol{x}^* \\ 0 & \text{otherwise} \end{cases}$ `// Intervention`

  (Rescaling) $\tilde{\boldsymbol{\alpha}}_t^{(h)} \leftarrow \text{Softmax}\left(\mathbf{a}_t^{(h)} + \mathbf{v}\right)$ for $h \in \mathcal{H}$

  $\ell_t \leftarrow \mathcal{M}(\boldsymbol{x}_{\leq t} \mid \tilde{\boldsymbol{\alpha}}_t)$ `// Rescaled Forward`

  $x_{t+1} \sim \text{Softmax}(\ell_t)$ `// Sampling next token`

end while

Key Design Choices:

- **Momentum smoothing** ($\gamma = 0.75$): Stabilizes relevance estimates across steps
- **Warm-up window** ($T_w = 16$): Uses last T_w tokens of input prompt to initialize $\mathbf{r}_T$
- **Intervention on ALL heads**: Propagates QRHead's relevance signal throughout the model
- **Log-space bias**: Adding $\log \beta$ to logits $\equiv$ multiplying attention by β before re-normalization

Efficient Implementation

Early Stopping of Aggregation

Observation: QRHead are primarily in the **middle layers** of transformer models.

For Qwen3-8B (36 layers):
QRHead distributed in layers **17–20**.

Strategy: During aggregation, only run up to the middle layers, then stop.

$\Rightarrow$ Saves $\sim$**60%** of computation per aggregation step.

Overhead Analysis

Setting: 128K input, 8K output tokens.

Component	FLOPs
Prefilling (128K)	Baseline
Decoding (4K output)	+3.2%
Decoding (8K output)	+6.4%
DYSCO overhead	
4K output	**+2.0%** total
8K output	**+3.8%** total

No cost during prefilling — DYSCO operates exclusively at decoding stage.

Baselines: How Does DYSCO Compare?

Approaches for Long-Context Inference

Method	How it works	Key Limitation
Vanilla	Standard decoding	Degrades with length
YaRN	Modifies RoPE for context extension	Extends window, doesn't improve reasoning
UniAttnS	Uniform scaling: $\mathbf{a}' = \mathbf{a}/\tau$	Rescales *all* tokens uniformly
RAG	Retrieve relevant chunks	External retriever; pipeline complexity
Prompt Compr.	Compress long prompts	Auxiliary LM; may lose info
DYSCO	Dynamic, **token-selective** rescaling	Small overhead ($\sim$4% FLOPs)

Key Distinction:

- UniAttnS: $\tilde{\alpha} = \text{Softmax}(\mathbf{a}/\tau)$ — **uniformly** sharpens all attention
- DYSCO: $\tilde{\alpha} = \text{Softmax}(\mathbf{a} + \mathbf{v})$ — **selectively** up-weights task-relevant tokens

Path Traversal: Diagnostic Results

Performance across context lengths (Table 1)

Path Traversal	4K	8K	16K	32K
Qwen3-32B				
Vanilla	80	67	32	21
+ UniAttnS	82	68	40	19
+ **DYSCO**	**79**	**71**	**46**	**33**
Qwen3-8B				
Vanilla	57	45	19	1
+ UniAttnS	59	46	25	2
+ **DYSCO**	**65**	**51**	**33**	**2**

Takeaways:

- DYSCO consistently improves across all context lengths
- Gains are **largest at longer contexts** (16K: +14 for both models)
- **Selective** > **Uniform rescaling** (DYSCO > UniAttnS)

Long-Context CoT Reasoning Results

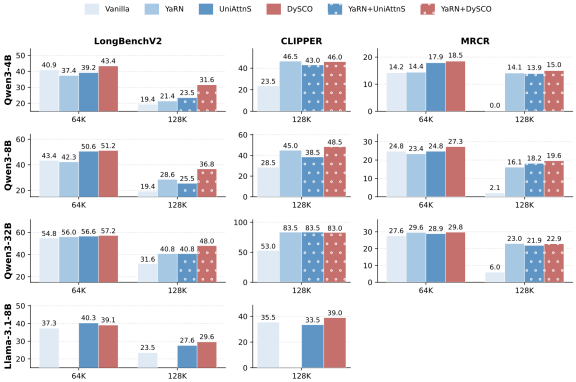


Fig 4: DYSCO (red) vs. Vanilla and UniAttnS across models/benchmarks.

Datasets: MRCR, LongBenchV2, CLIPPER at 64K and 128K.

At 128K context:

- DYSCO improves Qwen3-8B on MRCR by **+3.5** (22% relative)
- **+8.2** on LongBenchV2 (29%)
- **+2.5** on CLIPPER (10%)

UniAttnS gains are **smaller and inconsistent**.

Long-Context Recall Tasks (128K)

Direct-answer recall tasks (no CoT) — Table 2

	MRCR	HotpotQA	InfMC	InfQA	**Avg**
Qwen3-4B + YaRN					
Vanilla	16.7	37	50	27.1	32.7
+ UniAttnS	18.4	34	39	24.1	28.9
+ **DYSCO**	**19.0**	**39**	**54**	**31.3**	**35.8**
Llama-3.1-8B-Inst.					
Vanilla	17.2	46	61	37.1	40.3
+ UniAttnS	18.9	51	56	35.7	40.4
+ **DYSCO**	**18.6**	**52**	**62**	**38.4**	**42.8**

Observations:

- DYSCO improves recall even for **short-output** tasks (direct answer, no CoT)
- UniAttnS can **degrade** performance (e.g., InfMC for Qwen3-4B: 50→39)
- DYSCO's selective approach avoids such degradation

DYSCO vs. RAG & Prompt Compression

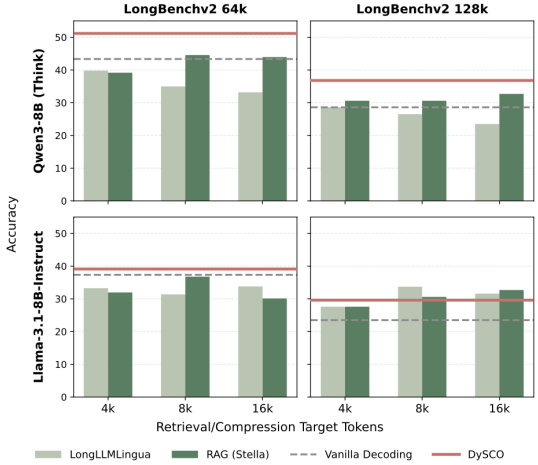


Fig 5: DYSCO (red line) outperforms RAG and LongLLMLingua on LongBenchV2.

Why DYSCO wins:

- **No external components** — no retriever, no compression model
- Operates on the **original input** without modification
- Compatible with **any input format** and task
- RAG/compression require explicit partitioning of input

neither RAG nor LongLingua yields consistent improvements at 64K.

Ablation Studies (Qwen3-8B)

Task	MRCR^{64K}		LongBV2^{64K}	Path^{16K}	InfMC^{128K}
Think	Yes	No	Yes	No	No
Vanilla	24.8	24.8	43.4	19	61
DYSCO	**27.3**	**26.3**	**51.2**	**33**	**66**
w/ RandomHead	26.6	24.2	47.0	20	63
w/ StaticScaling	22.5	23.1	47.5	23	64

Table 3: Head selection & rescaling strategy ablation.

Findings:

- **QRHead** > **Random heads**: More reliable relevance signals
- **Dynamic** > **Static rescaling**: Relevant tokens shift during generation

Hyperparameter Robustness (Table 4)

Config	p / K / β	MRCR	LBV2
Vanilla	–	16.1	28.6
Default	.975/8192/2.5	**19.6**	**36.8**
$p = 0.95$		18.4	31.7
$K = 2048$		19.6	38.0
$K = 4096$		19.0	32.7
$\beta = 3.0$		18.5	36.8
$\beta = 3.5$		18.1	31.7

DYSCO **consistently outperforms** vanilla across all hyperparameter settings.

Interplay: DYSCO × Chain-of-Thought

How does DYSCO interact with CoT reasoning? (Qwen3-8B, Fig. 6)

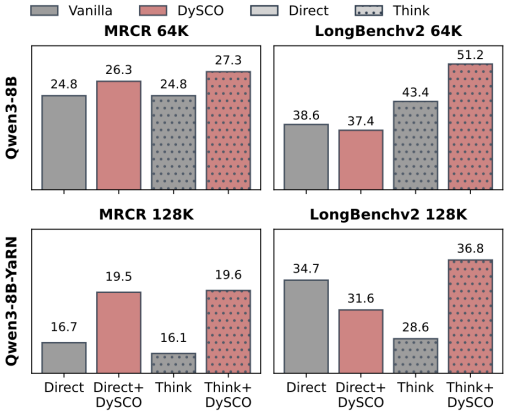


Fig 6: Think+DYSCO achieves the best results across all settings.

Key Finding:
DYSCO yields **larger benefits when combined with CoT reasoning**.

At 128K on LongBenchV2:
Think+DYSCO: **36.8**
vs. Vanilla Think: 28.6
(**+29%** relative)

CoT is inherently retrieval-intensive — DYSCO helps the model **refer back to relevant context** at each step.

Summary

DYSCO: Key Contributions

1. **New axis for long-context inference:** Improve **accuracy** at decoding time, not just efficiency.
2. **Training-free algorithm:** Three-stage decoding (Aggregation $\rightarrow$ Selection $\rightarrow$ Rescaling) that works on any off-the-shelf LM with $\sim$4% overhead.
3. **Mechanistic insight:** Retrieval heads (QRHead) provide reliable relevance signals even when overall performance degrades — DYSCO amplifies these signals across all heads.
4. **Strong empirical gains:** Up to 25% relative improvement on MRCR and LongBenchV2 at 128K. Outperforms RAG and prompt compression.

Core Takeaway:
Dynamic, **token-selective** attention rescaling $\gg$ uniform rescaling $\gg$ vanilla decoding.

Code: `https://github.com/princeton-pli/DySCO`

Future Directions

Future Directions:

- Combine DYSCO with **KV cache compression** — retrieval heads suggest which KV entries to keep
- Integrate with **agentic pipelines** and external scaffolding

Thank You! Any Questions?